

Llenguatges de Programació, FIB, 21 de juny de 2023

L'examen dura tres hores. Poseu el vostre nom a cada full. Contesteu cada problema en fulls diferents. Es valorarà la concisió, claredat i brevetat a més de la completesa i l'exactitud de les respostes. Feu bona lletra.

1 λ -càlcul en Python amb una IA

(2 punts)

Considereu aquest diàleg (minimament editat) entre una usuària i ChatGPT:

 Bon dia!

 Bon dia, amb què et puc ajudar?

 Ajuda'm amb la pràctica d'LP.

 Estaré encantat d'ajudar-te amb la pràctica d'LP (Llenguatges de Programació).

 Explica'm què és una beta reducció en el lambda càlcul.

 Una beta reducció en el lambda càlcul és una operació que permet avaluar expressions lambda mitjançant la substitució de les ocurrences d'una variable lligada en una funció per l'argument corresponent.

 Explica'm on cal aplicar una beta reducció a una lambda expressió en l'ordre normal.

 En l'ordre normal, cal aplicar una beta reducció a una lambda expressió en l'expressió més externa que encara no ha estat reduïda.

 Escriu-me en Python una funció

```
def normal_eval(e: Expr) -> Expr
```

que apliqui una β -reducció en ordre normal a una expressió en lambda càlcul utilitzant els tipus següents:

```
@dataclass
class Leaf:
    var: str

    @dataclass
    class App:
        lft: Expr
        rgt: Expr

    @dataclass
    class Abs:
        var: str
        body: Expr
```

```
Expr = Leaf | App | Abs
```

Pots suposar que no cal aplicar cap α -conversió.

 Aquí tens una possible implementació de la funció *normal_eval* en Python que aplica una β -reducció en ordre normal d'una expressió en el càlcul lambda suposant que no cal aplicar cap α -conversió:

```
def normal_eval(e: Expr) -> Expr:
    match e:
        case Leaf(var):
            return e
        case App(lft, rgt):
            if isinstance(lft, Abs):
                return substitute(lft.body, lft.var, rgt)
            else:
                return App(normal_eval(lft), normal_eval(rgt))
        case Abs(var, body):
            return Abs(var, normal_eval(body))

def substitute(e: Expr, var: str, r: Expr) -> Expr:
    match e:
        case Leaf(v):
            if v != var:
                return e
            else:
                return r
        case App(lft, rgt):
            return App(substitute(lft, var, r), substitute(rgt, var, r))
        case Abs(v, body):
            return Abs(v, substitute(body, var, r))
```

 Gràcies.

 De res! Estic aquí per ajudar-te.

Doneu la vostra opinió humana i experta. Màxim suggerit: 12 línies.

2 APL

(2 punts)

APL és un llenguatge de programació d'arrays altament expressiu i concís, dissenyat als anys 1960s per a operacions matemàtiques. Una de les peculiaritats d'APL és l'ús de caràcters i símbols no habituals. Per exemple, el producte s'escriu $\times$ i no $*$ i la divisió $\div$ enlloc de $/$. També, l'ordre d'avaluació en APL és de dreta a esquerra, si no s'usen parèntesis. Per tant, $5 \times 6 + 7$ val 65, però $(5 \times 6) + 7$ val 37.

Aquest programa en APL calcula tots els nombres primers fins a 100:

```
R ← 100
( R ∈ R o . × R ) / R ← 1 ↓ ⍋ R
```

Entre d'altres, el programa anterior utilitza la funció ι (iota, una i sense punt) que retorna un vector amb els naturals d'1 fins al seu paràmetre (així, ιR retorna $[1,2,3,\dots,R]$).

Aquesta expressió en APL

```
⌈ / v
```

calcula el màxim d'un vector v . De forma semblant, aquesta expressió en APL

```
+ / v
```

calcula la suma dels elements d'un vector v .

A més, la funció $\neq$ davant d'un vector en retorna la seva mida.

L'operador $\sim$ (que recorda la mirada confosa d'un neòfit en APL) funciona així: $5 - 3$ val 2 i $8 \div 2$ val 4 però $5 - \sim 3$ val -2 i $8 \div \sim 2$ val 0.25.

Tenint en compte les informacions anteriors:

1. Quines funcions representen els operadors binaris $\lceil$, $/$ i $\sim$ en APL?
2. Quins serien els seus possibles equivalents en Haskell i en Python?
3. Escriviu una expressió en APL que calculi

$$2\max\left(a, \frac{3}{b+c}\right) - 1$$

sense utilitzar parèntesis.

4. Escriviu una expressió en APL que calculi el factorial d'un nombre n .
5. Escriviu una expressió en APL que calculi la mitjana d'un vector v .

3 Inferència de tipus

(2 punts)

Inferiu el tipus complet de la funció

mySort = sortBy (compare `on` fst)

sabent que

```
data Ordering = LT | EQ | GT
sortBy :: (a -> a -> Ordering) -> [a] -> [a]
compare :: Ord a => a -> a -> Ordering
on :: (b -> b -> c) -> (a -> b) -> a -> a -> c
fst :: (a,b) -> a
```

Per a fer-ho, dibuixeu el seu arbre de sintàxi, etiqueteu els nodes amb els seus tipus i genereu i resoleu metodològicament les restriccions de tipus i classes. Expliqueu els passos realitzats.

4 Raonament equacional

(2 punts)

Aquestes són algunes funcions predefinides de Haskell:

replicate :: **Int** → *a* → [*a*]

replicate 0 *x* = [] (R1)

replicate *n* *x* = *x* : **replicate** (*n* − 1) *x* (R2)

foldr :: (*a* → *b* → *b*) → *b* → [*a*] → *b*

foldr *f* *z* [] = *z* (F1)

foldr *f* *z* (*x*:*xs*) = *f* *x* (**foldr** *f* *z* *xs*) (F2)

I aquesta és la definició de la classe mònada:

class **Monad** *m* **where**

(**>>=**) :: *m* *a* → (*a* → *m* *b*) → *m* *b*

(**>>**) :: *m* *a* → *m* *b* → *m* *b*

return :: *a* → *m* *a*

x **>>** *y* = *x* **>>=** _ → *y* (D)

que té aquestes lleis:

return *x* **>>=** *f* = *f* *x* (L1)

m **>>=** **return** = *m* (L2)

(*m* **>>=** *f*) **>>=** *g* = *m* **>>=** (\ *x* → *f* *x* **>>=** *g*) (L3)

Considereu aquestes dues funcions:

rep1 :: **Int** → **IO** () → **IO** ()

rep1 0 *io* = **return** () (11)

rep1 *n* *io* = **do** (12)

io

rep1 (*n* − 1) *io*

rep2 :: **Int** → **IO** () → **IO** ()

rep2 *n* *io* = **foldr** (**>>**) (**return** ()) (**replicate** *n* *io*) (2)

1. Digueu què fa *rep1* 4 (**putStrLn** "Hola").
2. Demostreu que *rep1* = *rep2*.
3. Demostreu que, per a tot parell de naturals *p* i *q*,

$$rep1 (p + q) io = rep1 p io \gg rep1 q io$$

Estructureu correctament les vostres demostracions i anoteu-ne cada pas realitzat amb el codi de l'equació utilitzada (R1, R2, ..., H1, ...).

5 Tipus i subtipus en C++ i Java

(2 punts)

1. Expliqueu què és un error de tipus, doneu la definició de llenguatge amb seguretat de tipus (*type-safety*) i digueu quan es pot comprobar la seguretat de tipus.
2. Considereu aquest programa en C++ (que utilitza els vells arrays de C i no els nous vectors de C++), amb un únic error de compilació a la darrera línia:

```
struct Animal { };  
struct Gat : public Animal { };  
struct Gos : public Animal { };
```

```
void f (Animal* animals[]) {  
    animals[0] = new Gos();  
}
```

```
int main() {  
    Gat* gats[] = { new Gat(), new Gat() };  
    f(gats);  
}
```

← *error: invalid conversion*

Tenint en compte l'error de compilació, expliqueu si el constructor d'arrays en C++ és covariant, contravariant o invariant.

3. Considereu ara aquest programa en Java (essencialment idèntic a l'anterior) però que compila sense cap diagnòstic:

```
class Animal { }  
class Gat extends Animal { }  
class Gos extends Animal { }
```

```
class Programa {  
  
    public static void f(Animal[] animals) {  
        animals[0] = new Gos();  
    }  
  
    public static void main(String[] args) {  
        Gat[] gats = new Gat[] { new Gat(), new Gat() };  
        f(gats);  
    }  
}
```

Tenint en compte que el programa compila perfectament, expliqueu si el constructor d'arrays en Java és covariant, contravariant o invariant.

4. Com pot ser que hom consideri que Java és un llenguatge amb seguretat de tipus quan la crida a *f(gats)* provocarà un error de tipus quan s'assigni un *Gos* en una llista de *Gats*? Altrament dit, com pot fer Java per evitar que es doni aquest error de tipus?